

NAME OF THE STUDENT : P. MUNI KARTHIK

REGISTRATION NUMBER : 192425061

COURSE CODE : DSA0306

COURSE NAME : NATURAL LANGUAGE PROCESSING

ASSESMENT NUMBER : CO2 AT-1

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The

program should accept the given input words, perform decomposition, classify each

suffix as inflectional or derivational, normalize the words to their common base form,

and display the parsed structure and normalized output in a tabular format.

PYTHON CODE :

```
words = ["connected", "connecting", "connection"]
```

```
print("{:<12} {:<10} {:<10} {:<15} {:<12}".format(  
    "Word","Root","Suffix","Type","Normalized"))
```

```
for word in words:
```

```
    if word.endswith("ed"):
```

```
        root = word[:-2]
```

```
        suffix = "ed"
```

```
        t = "Inflectional"
```

```
    elif word.endswith("ing"):
```

```

root = word[:-3]

suffix = "ing"

t = "Inflectional"

elif word.endswith("ion"):

    root = "connect"

    suffix = "ion"

    t = "Derivational"

print("{:<12} {:<10} {:<10} {:<15} {:<12}".format(

    word, root, suffix, t, "connect"))

```

OUTPUT :

main.py

Share

Run

```

1 words = ["connected", "connecting", "connection"]
2
3 print("{:<12}{:<10}{:<10}{:<15}{:<12}".format(
4     "Word", "Root", "Suffix", "Type", "Normalized"))
5
6 for word in words:
7     if word.endswith("ed"):
8         root = word[:-2]
9         suffix = "ed"
10        t = "Inflectional"
11    elif word.endswith("ing"):
12        root = word[:-3]
13        suffix = "ing"
14        t = "Inflectional"
15    elif word.endswith("ion"):
16        root = "connect"
17        suffix = "ion"
18        t = "Derivational"
19
20    print("{:<12}{:<10}{:<10}{:<15}{:<12}".format(
21        word, root, suffix, t, "connect"))

```

Output

Word	Root	Suffix	Type	Normalized
connected	connect	ed	Inflectional	connect
connecting	connect	ing	Inflectional	connect
connection	connect	ion	Derivational	connect

=== Code Execution Successful ===

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological

parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The

program should accept the given input words, identify the prefixes, suffixes, and base

forms, classify each transformation as inflectional or derivational, and display the

morphological breakdown and normalized root word in a tabular format.

PYTHON CODE :

```
words = ["unhappy", "happiness", "happily"]
```

```
print("{:<12} {:<8} {:<10} {:<10} {:<15} {:<10}".format(  
    "Word", "Prefix", "Base", "Suffix", "Type", "Root"))
```

```
for word in words:
```

```
    prefix = "-"
```

```
    suffix = "-"
```

```
    if word.startswith("un"):
```

```
        prefix = "un"
```

```
        base = "happy"
```

t = "Derivational"

```
elif word.endswith("ness"):
```

```
base = "happy"
```

suffix = "ness"

t = "Derivational"

```
elif word.endswith("ly"):
```

```
base = "happy"
```

suffix = "ly"

t = "Derivational"

```
print("{:<12} {:<8} {:<10} {:<10} {:<15} {:<10}".format(
    word,prefix,base,suffix,t,"happy"))
```

OUTPUT :

```
main.py  [Icons] [Share] [Run] Output

1 words = ["unhappy", "happiness", "happily"]
2
3 print("{:<12}{:<8}{:<10}{:<10}{:<15}{:<10}".format(
4     "Word", "Prefix", "Base", "Suffix", "Type", "Root"))
5
6 for word in words:
7     prefix = "-"
8     suffix = "-"
9
10    if word.startswith("un"):
11        prefix = "un"
12        base = "happy"
13        t = "Derivational"
14
15    elif word.endswith("ness"):
16        base = "happy"
17        suffix = "ness"
18        t = "Derivational"
19
20    elif word.endswith("ly"):
21        base = "happy"

=== Code Execution Successful ===
```

3. Given the input words: “played”, “player”, “playing”, design a stemming-based

preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a

search engine. The program should process the input words

“played”, “player”, “playing”;

by applying appropriate stemming rules to remove prefixes/suffixes where applicable,

identify the stem of each word, distinguish between grammatical inflections and word-

formation changes, and generate a structured output showing the original word,

extracted stem, removed affix (if any), transformation type (inflectional/derivational), and

the final normalized form suitable for indexing and information retrieval.

PYTHON CODE :

```
words = ["played", "player", "playing"]
```

```
print("{:<12} {:<10} {:<12} {:<15} {:<10}".format(  
    "Word", "Stem", "Affix", "Type", "Normalized"))
```

```
for word in words:
```

```

if word.endswith("ed"):
    stem = word[:-2]
    affix = "ed"
    t = "Inflectional"

elif word.endswith("er"):
    stem = word[:-2]
    affix = "er"
    t = "Derivational"

elif word.endswith("ing"):
    stem = word[:-3]
    affix = "ing"
    t = "Inflectional"

print("{:<12} {:<10} {:<12} {:<15} {:<10}".format(
    word,stem,affix,t,"play"))

```

OUTPUT :

```

1 words = ["played", "player", "playing"]
2
3 print("{:<12}{:<10}{:<12}{:<15}{:<10}".format(
4     "Word","Stem","Affix","Type","Normalized"))
5
6 for word in words:
7
8     if word.endswith("ed"):
9         stem = word[:-2]
10        affix = "ed"
11        t = "Inflectional"
12
13    elif word.endswith("er"):
14        stem = word[:-2]
15        affix = "er"
16        t = "Derivational"
17
18    elif word.endswith("ing"):
19        stem = word[:-3]
20        affix = "ing"
21        t = "Inflectional"

```

Word	Stem	Affix	Type	Normalized
played	play	ed	Inflectional	play
player	play	er	Derivational	play
playing	play	ing	Inflectional	play

```

=== Code Execution Successful ===

```

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb

inflection.

- Differentiate between regular and irregular inflectional patterns within the dataset.

- Ensure consistent mapping of all forms to a common root representation.

- Determine the state transitions, morphological breakdown, and normalized output for

each word.

- Develop a Python program that implements a finite-state morphological parser for the

input words "writes", "writing", "written".

The program should construct a finite-state

transition model to process regular suffixes and irregular verb forms, identify the

corresponding root word, classify each input as a regular or irregular inflection, trace the

sequence of state transitions during parsing, and display the state transition path,

morphological components, root form, and normalized representation for each input

word in a well-formatted output.

PYTHON CODE :

```
words = ["writes", "writing", "written"]
```

```
print("{:<12} {:<15} {:<18} {:<10} {:<12}".format(  
    "Word", "Pattern", "State Path", "Root", "Normalized"))
```


for word in words:

if word == "writes":

 pattern = "Regular"

 state = "Start->write->s"

elif word == "writing":

 pattern = "Regular"

 state = "Start->write->ing"

elif word == "written":

 pattern = "Irregular"

 state = "Start->write->en"

print("{:<12} {:<15} {:<18} {:<10} {:<12}".format(
 word,pattern,state,"write","write"))

OUTPUT :

```
main.py  [Icons]  Share  Run  Output
```

Word	Pattern	State Path	Root	Normalized
writes	Regular	Start->write->s	write	write
writing	Regular	Start->write->ing	write	write
written	Irregular	Start->write->en	write	write

```
6- for word in words:  
7-  
8-     if word == "writes":  
9-         pattern = "Regular"  
10-        state = "Start->write->s"  
11-  
12-     elif word == "writing":  
13-         pattern = "Regular"  
14-         state = "Start->write->ing"  
15-  
16-     elif word == "written":  
17-         pattern = "Irregular"  
18-         state = "Start->write->en"  
19-  
20     print("{:<12} {:<15} {:<18} {:<10} {:<12}".format(  
21         word,pattern,state,"write","write"))
```

=== Code Execution Successful ===

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-

based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.

- Handle variations in suffix patterns while preserving semantic consistency.

- Ensure uniform root extraction across all derived forms.

- Determine the stemming steps, intermediate forms, and final stem for each input word.

- Develop a Python program that implements the Porter Stemming algorithm for the input

words “relational”, “relation”, “relate”. The program should apply the Porter stemming

rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the

word meaning, and generate the final stem for each word. The output should clearly

present the original word, applied stemming rule(s), intermediate forms, and the final

normalized stem in a structured format suitable for document retrieval applications.

PYTHON CODE:

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
words = ["relational", "relation", "relate"]
```

```
print("{:<12} {:<20} {:<15}".format(  
    "Word","Applied Rule","Final Stem"))
```

```
for word in words:
```

```
    if word == "relational":  
        rule = "Remove -ational"
```

```
    elif word == "relation":  
        rule = "Remove -ion"
```

```
    else:  
        rule = "Remove -e"
```

```
    stem = ps.stem(word)
```

```
    print("{:<12} {:<20} {:<15}".format(  
        word,rule,stem))
```

OUTPUT :

```
print("{:<12} {:<20} {:<15}".format(  
    "Word","Applied Rule","Final Stem"))  
  
for word in words:  
    if word == "relational":  
        rule = "Remove -ational"  
  
    elif word == "relation":  
        rule = "Remove -ion"  
  
    else:  
        rule = "Remove -e"  
  
    stem = ps.stem(word)  
  
    print("{:<12} {:<20} {:<15}".format(  
        word,rule,stem))
```

Word	Applied Rule	Final Stem
relational	Remove -ational	relat
relation	Remove -ion	relat
relate	Remove -e	relat